

Assumptions

1. All airline staff may only hold one position at a time.
2. An airline staff member may perform only the actions associated with their specific role.
3. All airline staff can only work for one valid airline at a time.
4. Airline staff permissions are determined solely by role.
5. All tickets purchased by booking agents are only from airlines the agent is authorized to book with.
6. A booking agent must be authorized by at least one airline to purchase tickets to be considered a booking agent.
7. Airlines can have zero or more booking agents who can book their flight tickets.
8. All airlines use the same class naming system, but not all airlines offer the same selection of classes.
9. Tickets always belong to a customer, regardless of purchasing method.
10. Each flight is assigned to only one airplane.
11. Airplanes may serve multiple flights over time.
12. A booking agent may sell a ticket for a flight only if the flight is operated by one of the airlines for which the agent is authorized.
13. Airports can have many incoming/outgoing flights.
14. Each flight has exactly one departure airport and one arrival airport.
15. A ticket's class must be one of the classes offered by the airplane assigned to the ticket's flight.
16. Every ticket is purchased either directly or via a booking agent, but never both.
17. The total number of seats on an airplane must equal the sum of all seat classes in that particular plane.
18. Ticket entity has a "class_name" as a foreign key, referring to the "name" attribute of the Class entity.
19. Weak entity Flights has a composite key "flight_no" and "airline_name" (not listed). Flights will also have departure and arrival airport attributes as foreign keys in the relational schema.

Explanation

I chose to model ticket purchases with a ternary relationship between ticket, customer, and booking agent, while also including a binary relationship between customer and ticket. Through the ternary relationship "Agent_purchase", an agent may book a ticket on behalf of a customer, while the binary relationship "Direct_purchase" allows the customer to purchase a ticket for themselves. This is a disjoint process, and a ticket must be purchased through one or the other method, never both and never neither. This, though seemingly redundant, allows for two different channels of purchasing tickets as outlined in the project narrative.

Twist #2, seat classes, were implemented by adding an entity called "Class". Entity Airplane is related to Class via "Has_class", and this relationship has an attribute "class_seat_count", which is the number of seats of a particular class on a particular airplane. Every ticket bought will belong to a class, though not every class will be available for ticketing. This ensures that customers are buying specific class seats. I also added an attribute called 'ticket_id' to the Ticket entity, which is the unique identifier across all tickets, to aid in identifying tickets across different airlines.

Since airlines must own and operate planes/flights that are unique within the airline (per the narrative), Airplane and Flight are weak entities with identifying relationships to the strong entity Airline. It makes sense that Airplanes and flights cannot exist without an airline, thus I modeled them as weak entities. There are two price attributes: 'flight_price' under entity Flights refers to the overall advertised price of the flight, while 'ticket_price' under entity Tickets refers to the specific price of the class of the ticket, which is also the price that customers will pay for the ticket. This, though seemingly redundant, allows for greater pricing flexibility while also reflecting a real marketing strategy used by airlines.

A double relationship between entities Flight and Airport represents the departure and arrival airports of flights. Airport is modeled as a separate entity because it has an independent identity and participates in multiple relationships with Flights. For many entities, the owner entity's primary key isn't listed as an attribute of another (sometimes weak) entity in the ER diagram since it is implicitly included through their relationship. The key will be included as a foreign key and part of any weak entity's primary key in the relational schema.